

Client-Server Application & Network Latency Analysis

SENG 350 - Assignment 1



Authors: Bryan Welburn, Will Mohr
Date: 2026-10-08

Table of Contents

Table of Contents.....	2
Table of Figures and Tables.....	2
1. Introduction.....	3
2. System Architecture and Deployment.....	3
2.1 - Single Client.....	3
2.2 - Multiple Clients.....	4
3. Modifications to Application.....	4
3.1 - Client.....	4
3.2 - Server.....	5
4. Latency Measurements.....	5
4.1 - Experiment 1: Single-client.....	5
4.1 - Experiment 2: Dual-client.....	6
5. Latency Analysis and Architectural Reflection.....	7

Table of Figures and Tables

1. Figure 2.1: <i>Single-client deployment topology diagram</i>	3
2. Figure 2.2: <i>Dual-client deployment topology diagram</i>	4
3. Figure 4.1: <i>Single-client latency data (Machine 2)</i>.....	X
4. Figure 4.2: <i>Dual-client latency data (Machine 3)</i>	X
5. Figure 4.3: <i>Dual-client latency data (Machine 4)</i>	X

1. Introduction

In this assignment we deployed a distributed client-server application across multiple physical machines. In order to test the latency of our system, we developed a server-side program to calculate the fibonacci sequence from an input provided by the client. By using the latency data collected, we can calculate the server computational time, client invocation and response reception time, and the network & middleware transmission time. This data gives us insight into how our system performs when servicing multiple clients, and associated network and middleware overhead.

2. System Architecture and Deployment

2.1 - Single Client

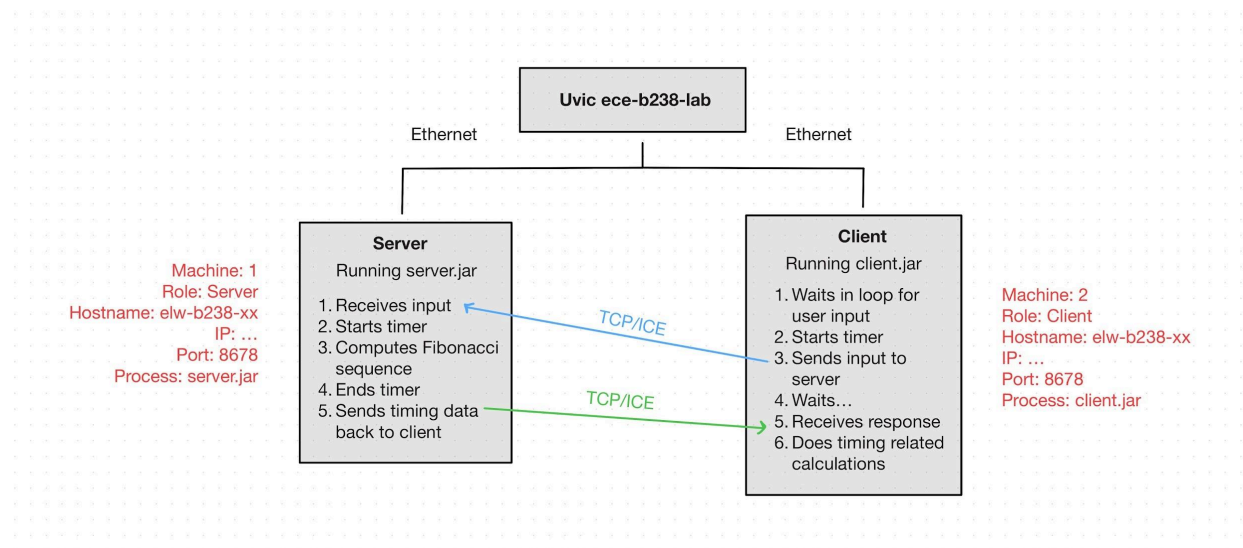
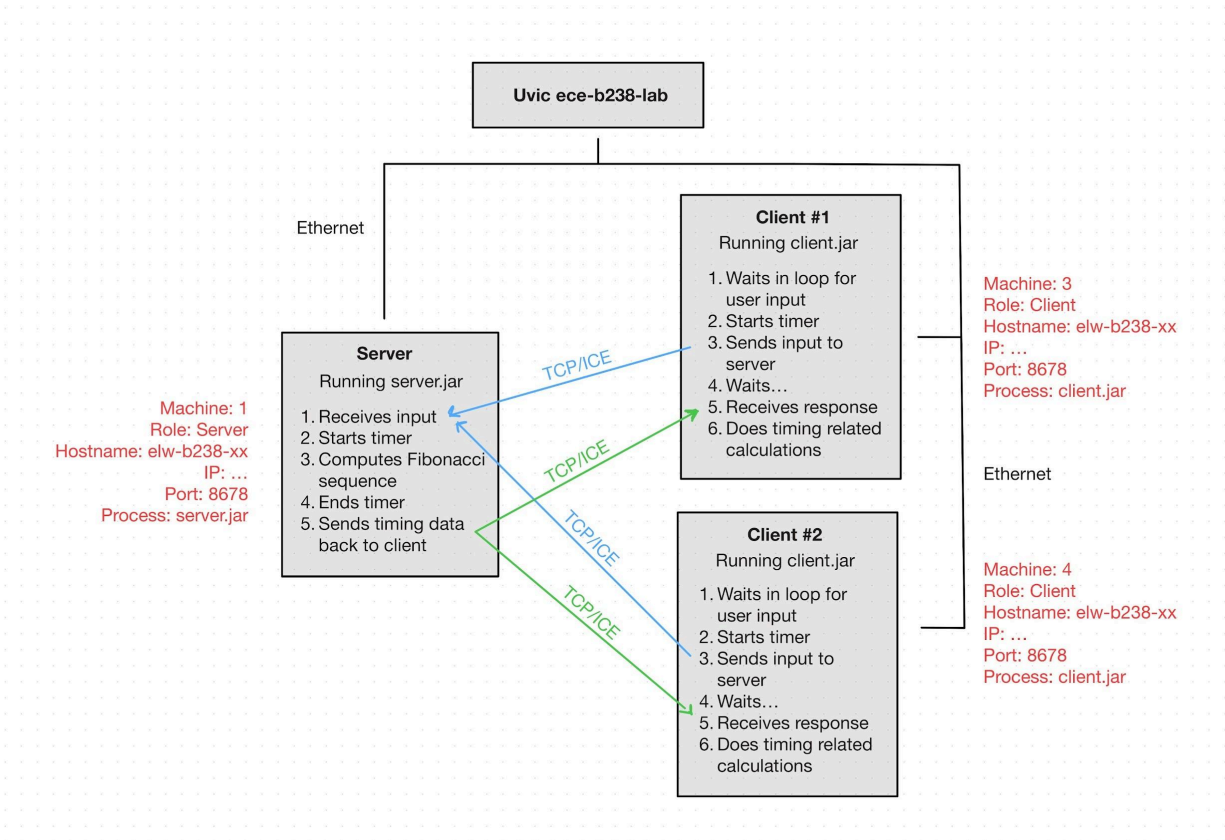


Figure 2.1: Single-client deployment topology diagram

2.2 - Multiple Clients



Dual-client deployment topology diagram

3. Modifications to Application

3.1 - Client

The client application was designed to wait in a continuous interactive loop. Once this loop is running, the user can input an integer into the terminal and send it to the server, where the Fibonacci sequence computation is performed. The client tags all of its messages with the client's own username and hostname (username:hostname:message). Right as this message is sent, the client starts a timer to track the total time until the server's response is received. After the client receives the response from the server (containing the total runtime of all the servers operations), the client uses the data to calculate the following:

- Server Service Execution Time
- Client Invocation & Response Reception Time
- Network & Middleware Transmission Time
- Total End-to-End Elapsed Time

3.2 - Server

The server application was configured to receive data using ICE from the client. After receiving the data, the server parsed the client's message, stripping out the username and hostname from the message. The server was designed to search through the given message to find the first instance of an integer. Due to stripping out all other information (including negative signs), any negative integers are interpreted as their positive representation.

The server code was designed using a recursive implementation of the Fibonacci sequence. This implementation only accepts integers n , where n is $0 \leq n \leq 45$. These restrictions are due to the Fibonacci sequence cannot accept inputs of $n \leq 0$, and a input of $n > 45$ will cause an integer overflow due to the output being too large. After computing the entire Fibonacci sequence for the given input, the server sends a response message to the client containing the total server runtime.

4. Latency Measurements

4.1 - Experiment 1: Single-client

Experiment 1 was used to measure the average latency of the server's Fibonacci sequence computation with a single client. In experiment 1, a value of 40 was sent from the client a total of 8 times. During this experiment there were no other clients attempting to communicate with the server.

Client 1 (Machine 2)

Server Service Execution Time (ms)	Client Invocation & Response Reception Time (ms)	Network & Middleware Transmission Time (ms)	Total End-to-End Elapsed Time (ms)
677	681	3	708
661	663	2	683
748	750	2	761
719	721	2	732
675	678	2	698
658	661	2	669
714	716	2	726
689	691	2	701
AVG: 692.625ms	AVG: 695.125ms	AVG: 2.125ms	AVG: 709.75ms

Figure 4.1: Single-client latency data (Machine 2)

4.1 - Experiment 2: Dual-client

During experiment 2 the average latency for the server's Fibonacci sequence was tested while the server was receiving concurrent requests from two separate clients. Similar to experiment 1, a value of 40 was sent from both clients 8 times each. The messages from the clients were sent at approximately the same time.

Client 2 (Machine 3)

Server Service Execution Time (ms)	Client Invocation & Response Reception Time (ms)	Network & Middleware Transmission Time (ms)	Total End-to-End Elapsed Time (ms)
700	703	3	731
652	1330	678	1341
681	1416	735	1426
676	677	1	690
664	666	1	676
672	674	2	681
694	1327	678	1338
647	1306	658	1317
AVG: 673.25ms	AVG: 1012.375ms	AVG: 344.5ms	AVG: 1025ms

Figure 4.1: Dual-client latency data (Machine 3)

Client 3 (Machine 4)

Server Service Execution Time (ms)	Client Invocation & Response Reception Time (ms)	Network & Middleware Transmission Time (ms)	Total End-to-End Elapsed Time (ms)
712	1413	700	1441
680	682	1	692
744	746	1	756
650	1326	676	1336

662	1324	661	1334
650	1314	663	1324
686	689	2	699
660	662	2	672
AVG: 680.5ms	AVG: 1019.5ms	AVG: 338.25ms	AVG: 1031.75ms

Figure 4.1: Dual-client latency data (Machine 4)

5. Latency Analysis and Architectural Reflection

The mean end-to-end latency determined by experiment 1 was about 710 ms. With experiment 2 the mean end-to-end latency was 1025ms for client 1 and 1032ms for client 2, giving an overall average of 1029ms. The difference between the average latencies between experiments can be explained by the architecture of the server application. The program runs on a single logical thread which means it can only process one fibonacci sequence at a time. This means that which ever client's request reaches the server second will be forced to wait for the other client's request to finish. This is why the largest increase in latency was observed to be with the "Network and Middleware Transmission Time" metric. This increase in time is caused by the second client request packets waiting in a network queue before being received by the server application. Without modifications to the architecture of the server program, this system would not scale well at all.